EnduroSense: presentation guide

The project in the order you will present it, the numbers to know, and short answers to the questions a panel is most likely to ask. The full version, with every detail, is `EnduroSense_Study_Guide_Full.pdf`.

How to present so the panel has little left to ask

1. **Say the limits before they do.** One drone, one battery type, short flights, a small test set. Said by you, it is honesty; found by them, it is a weakness.
2. **Show the misses yourself.** Two of four criteria were not met. Put them on a slide and explain them. A panel rarely digs into a failure you have already explained well.
3. **Answer "how do you know?" in advance.** Mention the locked test set and the criteria written first in one sentence, early.
4. **Keep the answers short.** One sentence of answer, one of evidence. Stop. Long answers invite follow-ups.
5. **If you do not know:** "That was not tested. What I can say is …" and give the nearest thing you did measure.

Every number is from the locked test set unless it says "development".

Part 1. The two-minute version

The brief asks for the remaining flight time of a UAV battery. Minutes are a weak basis for a decision, because they depend on what the drone does next. So I predict two energies: what the battery can still give above a safe reserve, and what a planned mission needs. Each comes with a range, checked on data the models never saw, and together they give a probability of success. A mission is approved at 95% or more.

I used 209 real flights of a DJI Matrice 100. Nobody recorded which battery was used, so I rebuilt that from resting voltages and split the data by battery, with 16 batteries locked away from the start.

I compared the five algorithms in the brief. The final battery model is five GRUs averaged: 2.2 Wh error on unseen batteries, against 2.9 for the best method without machine learning. The mission model is physics corrected by XGBoost, with 3.1% error.

On the locked test data, the brief's minutes-left rule approved 12.4% of missions that would have cut into the reserve. Mine approved 0.61%, and none at take-off. The price is caution: it refuses about a quarter of missions that would have been fine, mostly close calls.

I set four success criteria beforehand. Two were met. Two were not: the mission model was 5.7% on routes it had never seen, and its ranges were too narrow. I reported them as they are.

My guide then asked for two extensions, built as simulations on the real batteries and missions: a fleet planner that weighs priority, deadline, payload, charging and the probability of success, and a choice of where to process a task's data. Both met their criteria.

Part 2. The project, in presentation order

1. The problem and the idea

The brief: predict a UAV battery's remaining flight time, and compare Linear Regression, Random Forest, XGBoost, LSTM and GRU. Both were done. Minutes left is still predicted (0.8 min error).

Why minutes are not enough:

- **They depend on what the drone does next.** Minutes = energy left ÷ power now. Models with energy errors from 2.6 to 4.5 Wh all had minutes errors of 0.87 to 1.02 min: a better battery model barely helps.
- **One number hides how sure it is.** Two batteries can both show "30 Wh". One is 27 to 32 Wh, the other 16 to 39 Wh. For a 25 Wh mission, one is safe and one is not.

The idea: answer "can this battery fly this mission and keep a safe reserve?"

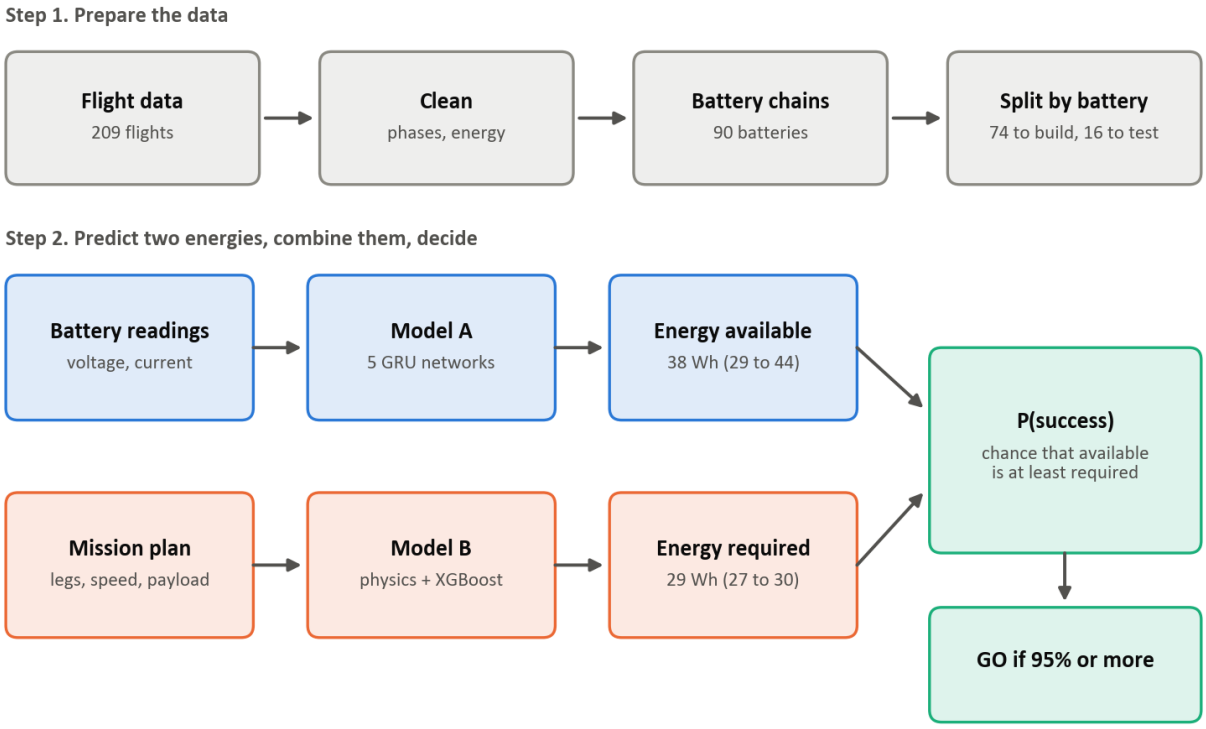


Figure 1. The whole pipeline. Example numbers are from the worked example in section 9.

- **Energy, not minutes:** energy adds up. A mission's energy is the sum of its parts, and the battery loses exactly that.
- **The reserve:** 22.6 V at rest (3.77 V per cell). "Energy available" means above it. **Unsafe = ending inside the reserve, not a crash.**
- **Settings:** reserve 22.6 V, threshold 0.95, ranges at 90%. I chose them and my guide left them to me. Each is one line in `config.yaml`; a change takes a 30-minute rerun.

2. The data and what was fixed

Data: Carnegie Mellon University, "Data Collected with Package Delivery Quadcopter Drone" (Rodrigues et al., 2021, public). DJI Matrice 100, TB47D battery (99.9 Wh). **209 flights, 257,896 readings** at about 8

per second, 18 days in 2019. Planned grid: 5 speeds × 3 payloads × 4 altitudes. 182 flights on one route (R1: a 3-leg loop of 140, 198 and 119 m). A typical flight: 3 minutes, 20 Wh.

Why it: real flights, voltage and current recorded (so energy is exact), a planned grid, public. **Its limits:** one drone, one battery type, mostly one route, short flights, and **no record of which battery was used**.

Problem found	Fix
Start times stored as text: "9:22" sorted after "10:05", 7 days out of order	Sort by real timestamp; a test checks it
4 mis-recorded start times	Corrected in code with voltage evidence; **raw data never edited**
Documentation has the vertical-speed sign backwards	Data shows up is positive (correlation +0.77); used that
Altitude drifts up to 14 m after landing	Current under 2 A = motors off = on the ground
76 sensor glitches (0.03%)	Median of the 5 surrounding readings, flagged

Energy = voltage × current × time, summed reading by reading. Every reading is also labelled with a flight phase: ground, climb, cruise, hover, descent.

3. Battery chains and the locked test set

Why it matters: readings from one battery are near-copies. One battery in both training and test leaks the answer and makes results look better than they are.

Rebuilding the batteries: at rest, voltage shows the charge. If the next flight starts within −0.10 to +0.40 V of where the last ended, it is the same battery; a big jump means a fresh one. Lopsided on purpose: a resting battery recovers, it does not drop.

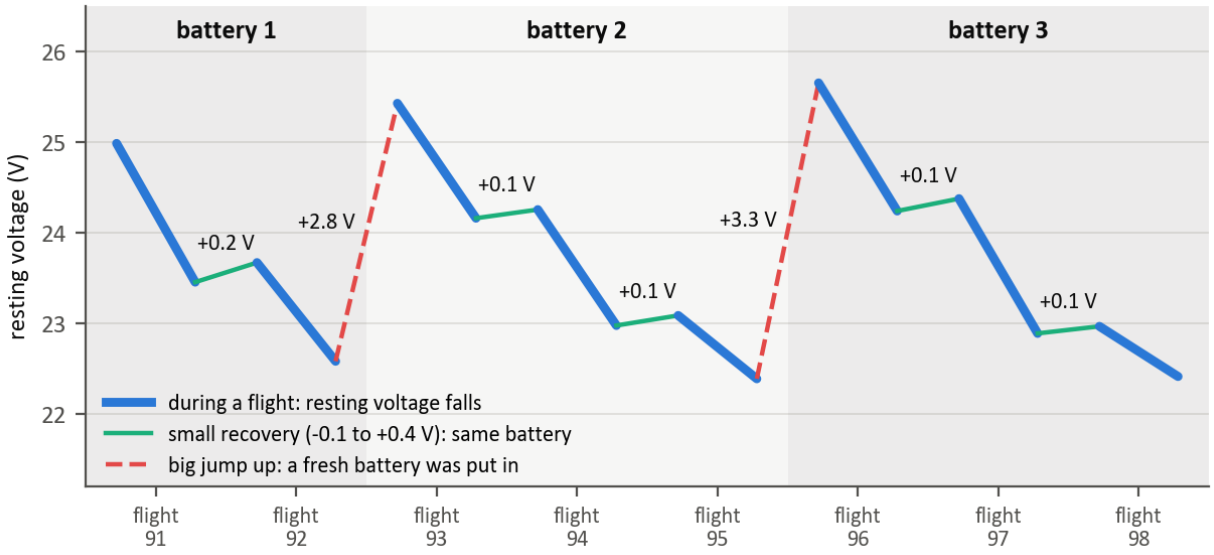


Figure 2. Eight flights on one day: small recoveries link flights into one battery; big jumps are battery swaps.

Result: 90 batteries ("chains"); other reasonable windows give 87 to 92.

The split, by battery:

- **16 batteries (40 flights) locked away before any model was built.** Chosen as the best-balanced of 5,000 random splits; the longest route (R6) was forced in as an unseen distance.

- **The lock:** the split file is hashed, the code refuses to read test data outside the final evaluation, and every opening is logged with a fingerprint of the code. The log has three entries: the evaluation, a rerun that changed only reporting (identical numbers), and the extensions.
- **74 batteries (169 flights) for development,** in 5 folds by battery. Cross-validation: train on 4, predict the 5th, so every battery is predicted by a model that never saw it.

4. Model A: energy available

The label: for a battery flown down to the reserve, the energy drawn until its resting voltage reached 22.6 V (E_reserve). At any moment: remaining = E_reserve − energy drawn so far. The crossing is found along a fitted discharge curve (development data only); a straight line was biased by 3.3 Wh. The labels themselves are uncertain by 1.5 to 2.2 Wh. Full batteries give 47 to 77 Wh above the reserve: that spread is what Model A must learn.

The inputs (25, all from the past only): voltage, current and power now; their averages and trends over 10 and 30 s; internal resistance and voltage sag; resting voltage before this flight and when first used; energy drawn; flight number on the battery; motors on or off. LSTM and GRU also read the last 60 s as a sequence (120 steps). A test changes every reading after time t and confirms no input at t changes.

Baselines ML must beat: fixed capacity (13.6 Wh error, reference only); energy counting (what a battery management system does); **voltage lookup** (re-read the charge from the resting voltage; the strongest non-ML method).

Algorithm	In one line	Development error
GRU	Reads the last minute in order, keeps a memory	**2.61 Wh**
LSTM	Same, with a more complex memory	2.65 Wh
Random Forest	Average of many decision trees	3.02 Wh
Linear Regression	Weighted sum of the 25 inputs	3.02 Wh
XGBoost	Trees built one after another, each fixing the last	3.26 Wh
Voltage lookup (no ML)		*3.22 Wh*

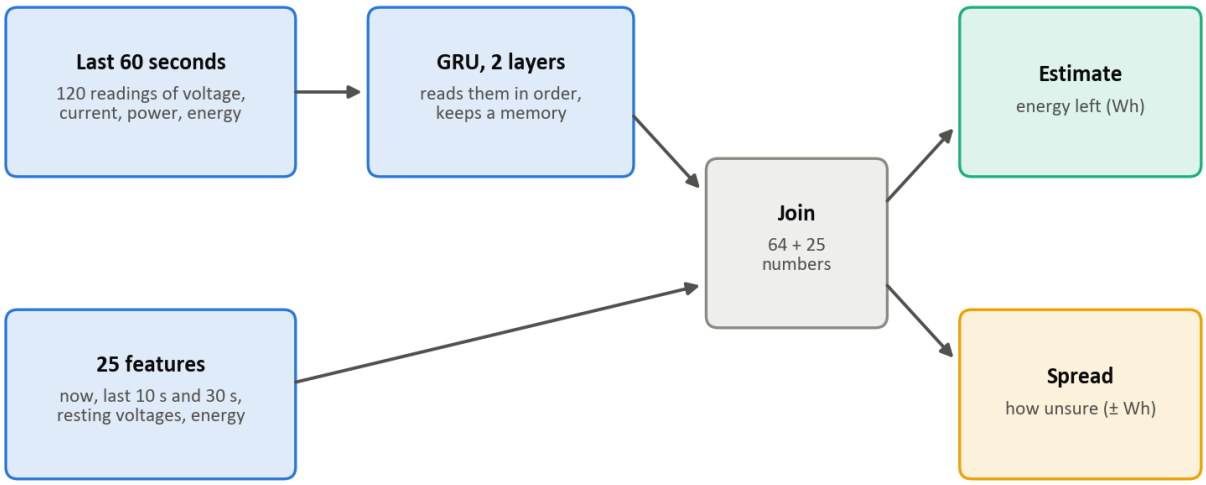


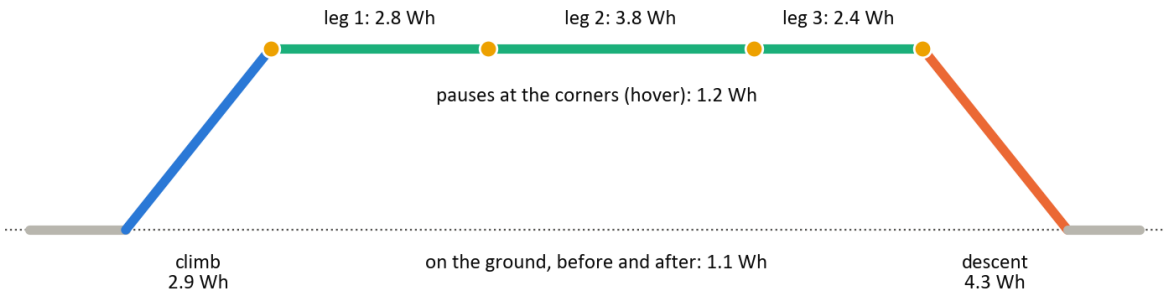
Figure 3. Inside the GRU model.

- **Why GRU and LSTM win:** they see how voltage *responds* to load over the last minute, which reflects that battery's health.

- **Why XGBoost does not:** with 49 development batteries, trees learn individual batteries. Splitting by battery exposes it; a random split would have hidden it.
- **Final model: five GRUs averaged.** On the test, a single GRU scored 2.81 Wh (worse than the baseline); five averaged scored **2.19 Wh** against the baseline's 2.90. Averaging is what made it dependable.
- **Speed:** 6 ms or less per prediction on a laptop; the GRU file is 0.18 MB.

5. Model B: energy a mission needs

Mission energy = climb + Σ legs (power × time) + hover between legs + descent + ground



climb 2.9 + legs 9.1 + hover 1.2 + descent 4.3 + ground 1.1 = 18.6 Wh

90% range: 17.7 to 19.5 Wh

Figure 4. Model B adds up the parts. Example: an R1-like loop at 8 m/s, 50 m, 250 g.

- **Per leg,** because every recorded flight flies the same loop: 587 legs of different lengths are the only way to learn distance.
- **Leg time** = distance ÷ speed + time to speed up and brake (0.5 s at 4 m/s, 4.4 s at 12 m/s).
- **Physics for power:** rotor momentum theory (a constant part + power to hold the weight up + drag rising with speed cubed). Airframe mass limited to the Matrice 100's real range.
- **"Physics-first":** physics (+ XGBoost on its leftover error) unless pure ML is more than 5% better. Development error **2.35%**. The pure "best model per part" choice under-predicted fast missions by 0.44 Wh, which is the unsafe direction.
- **Why physics:** on an altitude or payload never seen, tree models were 9 to 14% off; physics-first 2.5 to 3.1%.
- **No LSTM/GRU here:** a mission plan is a handful of numbers, not a sequence.
- **Warnings** outside the tested range: speed 4 to 12 m/s, payload 0 to 750 g, altitude 25 to 100 m, legs 12 to 334 m.

6. Honest ranges

Calibrated = a "90% range" holds the truth about 90% of the time on unseen data.

Model A's range:

1. Five GRUs. Each gives an estimate and a spread; where they disagree, the model is unsure.
2. The estimate and the spread are trained separately (training them together made the estimate worse).

3. **Conformal calibration:** on batteries the models never saw, measure how many "spreads" off the truth really was, and set the range from those measurements instead of a bell curve. Over-estimates were larger than a bell curve assumes, so the range reaches further down.
4. Every battery counts equally, because thousands of readings from one battery share one error.
5. No pre-flight resting voltage, no answer: the decision is NO-GO.

Model B's range: replay the real errors of every unseen flight (153 of them) on the new mission's parts; the spread of the totals is the range. One flight's errors stay together, and sorties on one battery share one error (independent errors gave "90%" ranges that held only 81 to 85%).

Development: Model A's 90% range held 89.5%; Model B's 91.5%. **Key finding:** Model A's error is mostly a fixed offset per battery. If it over-estimates a battery, it does so for that battery's whole life.

7. The decision and how it was tested

P(success) = the chance that energy available ≥ energy required, calculated from the two ranges on a fixed 400-point grid (same inputs, same answer). **GO if ≥ 0.95.**

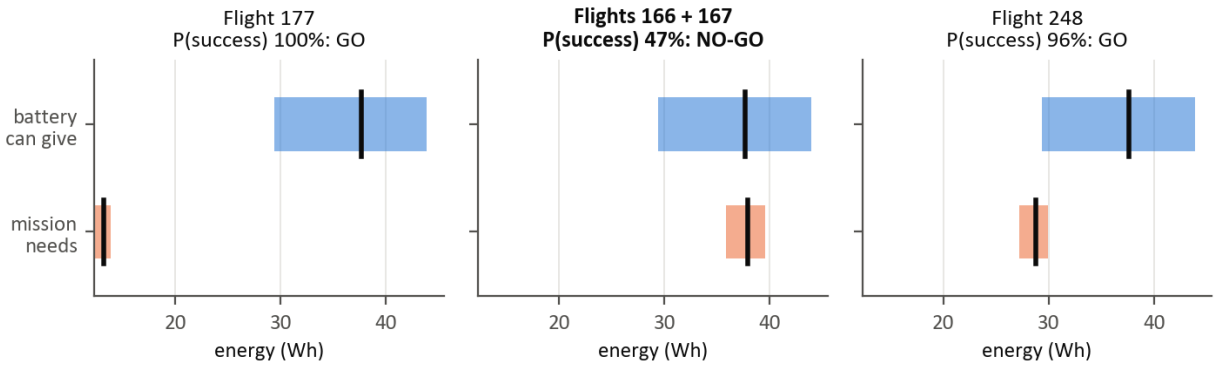


Figure 5. One battery, three missions: where the ranges overlap, the probability drops.

Testing without flying: pair a real battery state with a real recorded mission. Both true energies are measured, so the right answer is known. 20,000 pairs; missions include real back-to-back sorties.

Rule	Approves when
P1 Minutes left (the brief)	minutes left ≥ mission duration (given the *true* duration)
P2 Best estimates	estimated available ≥ estimated needed
P3 EnduroSense	P(success) ≥ 0.95
P4 Oracle	the truth says yes

Two kinds of mistake: an **unsafe approval** (approving a mission that would cut into the reserve) and a **wasted refusal** (refusing one that would have been fine). Being stricter trades one for the other, so P1 and P2 were also given safety margins tuned on development data.

Fleet simulation: 4 drones, 40 tasks a day, 200 days. Real batteries and real missions; only the pairing is simulated.

8. Final results on the locked test set

Order of events: rebuilt everything from raw data (matched exactly), wrote down the criteria, rehearsed on development data, then opened the test set and changed nothing.

#	Criterion	Result	Evidence
1	Model A beats both non-ML baselines	**Met**	2.19 Wh against 2.90 and 4.21
2	Model B within 5%, including unseen routes	**Not met**	3.1% overall; 5.7% on 7 unseen-route flights
3	90% ranges hold the truth 88–92% of the time	**Not met, within sampling noise**	Model A 93.8%, Model B 80.0%
4	Better decisions than minutes-left and best estimates	**Met**	0.61% unsafe approvals against 12.4% and 3.6%

Rule	Unsafe approvals	Wasted refusals	Unsafe at take-off
Minutes left (the brief)	12.4%	3.3%	12.8%
Best estimates	3.6%	3.9%	4.3%
Best estimates + 7 Wh margin	0.55%	24.4%	0.00%
EnduroSense	**0.61%**	24.3%	**0.00%**

- **1 in 8 against 1 in 160.** All 71 unsafe approvals involve the same two batteries Model A over-estimated (chain 70 by 7.4 Wh, chain 86 by 5.1 Wh).
- **The refused good missions are close calls:** typical true margin 3.8 Wh.
- **Minutes-left cannot be saved by a margin:** made as safe, it refuses 91% of good missions.
- **A tuned margin on best estimates did about as well overall.** The probability refused fewer at take-off (25% against 32%) and needs no tuning.
- **Model B misses:** all 7 unseen-route flights were over-predicted (the safe direction); the longest route's cruise is about 820 m against 443 to 568 m in training.
- **Fleet:** 12.0 unsafe missions per 100 with minutes left, 0.73 with EnduroSense, for about 10% more battery swaps.
- **Is P(success) honest?** Missions rated 95–99% succeeded 94% of the time: slightly over-confident, because of the two over-estimated batteries.

9. The worked example (know it cold)

Battery chain 86, before take-off. The model says 38 Wh above reserve; 33 Wh were really there. The minutes-left rule says GO to all three missions.

Mission	Really needed	P(success)	EnduroSense	What happened
Flight 177	14 Wh	100%	GO	Fine
Flights 166 + 167, back to back	38 Wh	47%	**NO-GO**	Would have gone 5 Wh into the reserve
Flight 248	28 Wh	96%	GO	Fine, 5 Wh to spare

Chosen from the test data by a fixed rule, not by hand.

10. The extensions requested by the guide

Both are simulations on top of the finished models. Real: the batteries, missions, ranges and hover power (539 W, measured). Simulated: task arrivals, priorities, deadlines, payload limits, chargers, and all radio and server settings. Criteria written first; one logged run on the 8 unseen batteries; all three criteria met; nothing earlier changed.

1. Fleet planner. 4 drones (payload limits 750, 500, 500, 250 g), 100 tasks in 8 hours; high priority due in 30 min, normal 90, low 240; 2 chargers at 100 W. Every minute it matches all waiting tasks to all free

drones at once (Hungarian algorithm). A pair is allowed only if the payload fits, the task finishes on time and P(success) ≥ 0.95; priority first, then deadline, then best-fitting battery.

Policy	Weighted tasks on time	High priority on time	Unsafe per 100
First come, minutes left	67%	53%	11.1
First come, EnduroSense check	71%	62%	0.62
Planner, best estimates	84%	96%	4.46
Planner, EnduroSense	**84%**	**95%**	**0.61**

Planning gives the speed; the probability gives the safety.

2. Where to process a task's data: on board, on an edge server at the depot, or on another drone. **Hovering while waiting is the real cost:** 30 s of hovering is 4.5 Wh; 30 s of computing is 0.13 Wh. The energy-aware rule measures the link and the server queue on site and picks the cheapest option that meets the deadline: **95.6% of jobs on time at 2.05 Wh per task**, against 90.2% at 2.60 Wh always on board. Weak link: it moves work on board; strong link: it offloads more.

11. Why the results can be trusted

Safeguard	In one line
Split by battery	No battery in both training and test
Locked, logged test set	Chosen before modelling, gated in code, every opening logged with a reason
Criteria written first	They could not be adjusted after seeing results
One-command rebuild	`python scripts/run_all.py --final`, 30 min; a from-scratch rebuild matched byte for byte
166 automatic tests	Including causality, the split, and every table's arithmetic
Mutation testing	Plant a bug on purpose; check a test catches it
Six verification passes	Each stage re-checked as a sceptic, key numbers recomputed separately

Mistakes found and fixed (know two): flights sorted by text time (7 days out of order); summary figures that included test data (no model used it); a first Model B that under-predicted fast missions; sorties on one battery treated as independent; a write-up comparing test error with a development figure from different data.

Part 3. Numbers to know by heart

Flights / readings / batteries	209 / 257,896 / 90
Test set	16 batteries, 40 flights (11 with an energy label)
Battery pack / usable above reserve	99.9 Wh / about 65 Wh
Reserve / threshold	22.6 V at rest / 0.95
Unsafe approvals: minutes-left / best estimates / EnduroSense	**12.4% / 3.6% / 0.61%** (0.00% at take-off)
Good missions refused	24% (close calls)
Model A error	**2.19 Wh** (best non-ML 2.90 Wh)
Model B error	**3.1%** (2.6% trained route, 5.7% unseen routes)
90% ranges held the truth	Model A 93.8%, Model B 80%
Fleet: unsafe per 100	12.0 against 0.73
Criteria	2 of 4 met; extensions 3 of 3
Speed / tests / rebuild	≤ 6 ms / 166 / 30 min

Part 4. The questions most likely to come up

Q. Did you still do what the brief asked?

Yes. Minutes left is predicted (0.8 min error) and all five algorithms are compared on error, speed and size. The energy decision sits on top.

Q. Why is energy better than minutes?

Energy adds up and minutes do not. Minutes depend on the power drawn right now; our models' energy errors ranged from 2.6 to 4.5 Wh but their minutes errors all sat near 0.9 min.

Q. Why split by battery? Why not randomly?

Readings from one battery are near-copies; a random split tests on batteries already seen. Split randomly, XGBoost looks good; split by battery, it does not beat the simple baseline.

Q. The dataset does not say which battery was used. How do you know?

Resting voltage. Between flights on one battery it stays or recovers slightly; a big jump means a swap. 90 batteries; other reasonable settings give 87 to 92.

Q. How do we know you did not tune on the test set?

It was chosen before modelling, the code refuses to read it outside the final evaluation, every opening is logged with a code fingerprint, and the criteria were written first. Nothing was adjusted after opening it.

Q. Why does the GRU win, and why five of them?

It reads how voltage responds to load over the last minute, which shows that battery's health. One network depends on its random start: a single GRU scored 2.81 Wh on the test; five averaged scored 2.19.

Q. Why did XGBoost do poorly? It usually wins on tables.

With 49 development batteries it learns individual batteries, which does not carry over. Splitting by battery exposes that.

Q. Is 2.19 Wh good?

About 3% of usable energy, 0.71 Wh better than the best non-ML method, and close to the noise in the labels themselves (1.5 to 2.2 Wh).

Q. Why physics in Model B, and not LSTM or GRU?

Physics extends to conditions never seen: 2.5 to 3.1% error on an unseen altitude or payload, against 9 to 14% for trees. A mission plan is a few numbers, not a sequence, so there is nothing for a sequence model to read.

Q. Model B missed on unseen routes. Explain.

5.7% against a 5% target on 7 flights, all over-predicted, which is the safe direction. The longest route has about 820 m of cruise against at most 568 m in training. I reported it and did not adjust the model, because re-scoring on the same test set would be dishonest.

Q. What is a calibrated range, and how do you get it?

A range that is as reliable as it claims. I measure how wrong the model really was on batteries it never saw and set the range from those errors, with every battery counted equally.

Q. Your Model B range held the truth only 80% of the time. Failure?

It missed the band. With 40 flights, 90% is still inside the sampling interval (65 to 92.5%), so it is "not met, within sampling noise". But it is a real weakness: too narrow on new data, mostly at 100 m altitude, already flagged in development.

Q. How is P(success) computed? Are the two errors independent?

From the two ranges on a fixed 400-point grid. Not quite independent (correlation 0.43 on the test), but Model B's error is about four times smaller, and the link is in the safe direction.

Q. How can you test decisions without flying?

Pair a real battery state with a real recorded mission. Both true energies were measured, so the right answer is known.

Q. It refuses 24% of good missions. Too cautious?

They are close calls: typical true margin 3.8 Wh. The threshold is a dial: at 90% it refuses 18%, at 99% 34%. A better battery model would refuse fewer.

Q. Is the probability better than adding a margin to the estimates?

Over all cases, about the same (0.55% against 0.61%). Before take-off it refused fewer good missions (25% against 32%), and it needs no tuning. The big gain is over minutes-left.

Q. Two of four criteria failed. Why trust this?

Because they were written first and the misses are reported as misses. The two that matter most were met: the battery model beats the baselines, and decisions are far safer than the brief's rule.

Q. What is the biggest weakness?

Per-battery risk: if the model over-estimates a battery, it does so every flight. All unsafe approvals came from two such batteries. The fix is learning each battery's capacity as it flies.

Q. Your extensions are simulated. Are they meaningful?

The batteries and missions in them are real; the schedule and radio settings are simulated, and I say so. They show that the decision rule plugs into scheduling and edge computing and keeps its safety: planner with best estimates 4.46 unsafe per 100, with EnduroSense 0.61.

Q. A cold day, an old battery, a different drone?

Not tested. Internal resistance and voltage sag respond indirectly to cold and age, but the ranges were calibrated on this data. A new drone or battery needs its own data and recalibration; the method carries over.

Q. Would it run on a real drone?

Model A needs only voltage and current once a second, takes 6 ms or less on a laptop, and is 0.18 MB per network. Model B and the decision are quick calculations. On-board timing was not done.

Part 5. What not to claim

Do not say	Say
"The probability beats a safety margin"	"It matched a tuned margin overall, refused fewer good missions at take-off, and needs no tuning"
"Model B is accurate to 3%"	"3.1%; 2.6% on the trained route, 5.7% on unseen routes"
"The ranges are calibrated"	"Model A's held 93.8%; Model B's were too narrow at 80%"
"GRU is the best algorithm"	"Five GRUs averaged was best; a single GRU did not beat the baseline on the test"
"The test set was used once"	"Evaluated once, rerun once with identical numbers, and used once for the extensions; all logged"
"It works for any drone"	"Built and tested on one drone and one battery type"
"Unsafe means a crash"	"Unsafe means ending inside the safety reserve"
"The planner was tested on real schedules"	"The schedules are simulated; the batteries and missions are real"

Part 6. Demo order (about 7 minutes)

Start `streamlit run app/streamlit_app.py` beforehand and click every page once. Keep τ at 0.95 and "Show what was measured" off.

1. **Three missions, one battery (2 min).** Minutes-left says GO to all three; EnduroSense refuses the middle one (47%). Switch on "Show what was measured": it would have gone 5 Wh into the reserve.
2. **Mission check (2 min).** Pick a battery before take-off; drag the distance per leg from 300 to 600 m and watch P(success) fall and flip to NO-GO while minutes-left still says GO. Point at the out-of-range warning.
3. **Fleet (1 min).** 12 unsafe missions per 100 with minutes-left, under 1 with EnduroSense, about 10% more battery swaps.
4. **Fleet planner (1 min).** Criteria all met; 95% of urgent tasks on time against 62% first-come; safety from the probability.
5. **Results (1 min).** Read out the two criteria not met, before anyone asks.

If the dashboard fails, the slides have every chart.